System Manual

WebXR-Enabled Digital Heritage Platform

NHCP Museo ng Libingan sa Ilalim ng Lupa ng Nagcarlan

Document Control

Item	Information
System name	WebXR-Enabled Digital Heritage Platform
Institution	NHCP Museo ng Libingan sa Ilalim ng Lupa ng Nagcarlan
Document type	System Manual
System version	Version 1.0 – Initial Production Release
Document version	Version 1.0
Date prepared	September 11, 2026
Prepared by	EW01
Maintained by	Authorized Museum Administrator
Production URL	https://nhcp-digital-heritage.vercel.app

1. Introduction

1.1 Purpose

This System Manual provides the technical information needed to install, configure, deploy, administer, troubleshoot, update, back up, and maintain the WebXR-Enabled Digital Heritage Experience Platform for the NHCP Museo ng Libingan sa Ilalim ng Lupa ng Nagcarlan. Unlike the User's Manual, this document is intended for administrators, developers, maintainers, information-technology personnel, and future researchers responsible for the technical operation of the system.

1.2 Intended Readers

Readers performing installation or maintenance should have basic knowledge of Python, Flask, HTML, CSS, JavaScript, PostgreSQL, Supabase, Git, web hosting, and environment-variable configuration. Access to production resources must be limited to personnel authorized by the museum.

1.3 Scope

This manual covers the architecture, requirements, dependencies, folder structure, database and storage, local installation, configuration, administration, deployment, backup, maintenance, security, testing, and troubleshooting of the completed platform.

2. System Overview

The platform is a web-based digital heritage system that combines museum information, digital exhibits, educational resources, an interactive site map, a 360-degree virtual tour, augmented reality, three-dimensional objects, WebXR environments, visitor services, and administrative content management.

Public visitors can explore heritage content without an account. Registered users can access bookmarks, profile settings, guided-tour bookings, community participation, feedback, Heritage Keeper progress, collected items, and achievements. Authorized administrators can manage users, bookings, content, submissions, feedback, notifications, analytics, virtual tours, site-map information, and account security.

3. System Architecture

The system uses a layered client-server architecture. The client layer contains the public, account, immersive, and administrative interfaces. The application layer uses Python and Flask for routing, Jinja rendering, authentication, validation, business rules, and service integrations. The data layer uses Supabase PostgreSQL, Supabase Storage, packaged assets, and WebXR scene archives. External services provide optional Google OAuth, SMTP email, ElevenLabs narration, hosted sentiment analysis, and compatible document preview.

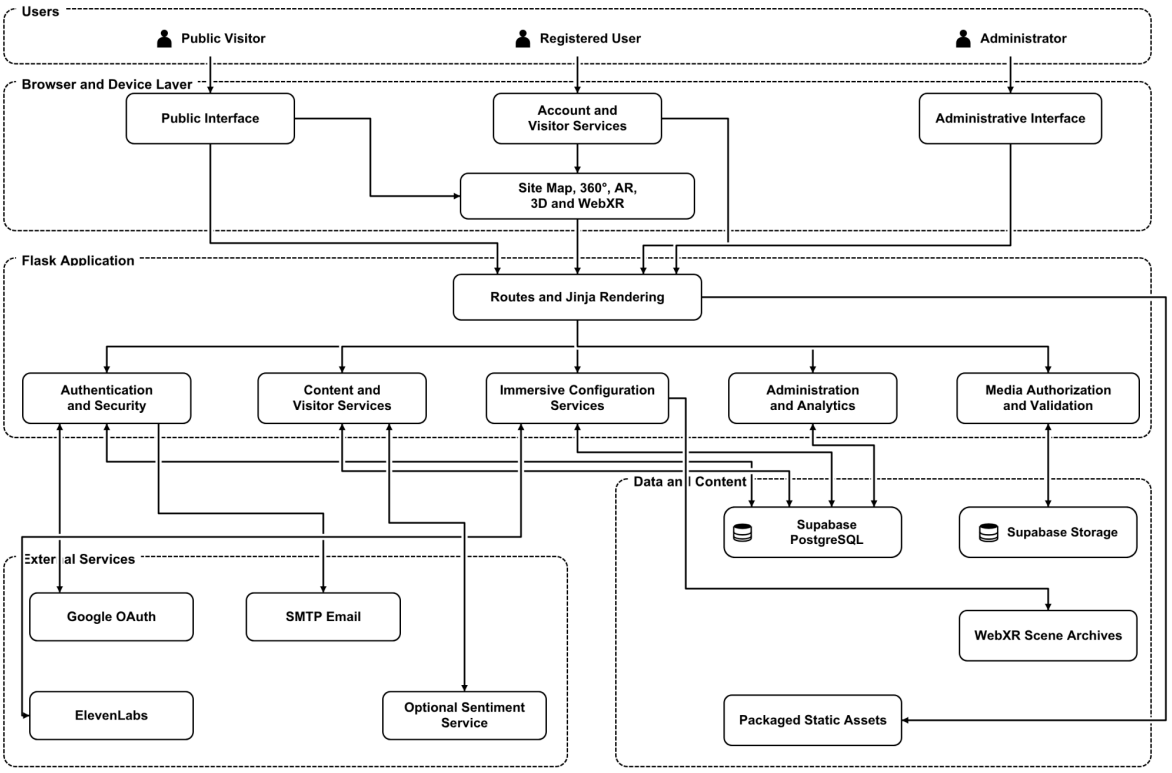


Figure 3.1 System Architecture of the WebXR-Enabled Digital Heritage Platform.
This figure shows the interaction among users, browser interfaces, Flask services, data resources, and external services.

3.1 Security Architecture

Public-user and administrator authentication are separate. The application uses password hashing, email verification, administrator OTP, signed sessions, role checks, CSRF tokens, rate limits, file validation, security headers, protected server credentials, and database Row Level Security. The Supabase service-role key remains on the server and must never be included in browser code.

4. Hardware Requirements

Table 4.1 Hardware Requirements

Device	Minimum	Recommended
Development computer	64-bit processor, 8 GB RAM, WebGL-capable graphics, 5 GB storage, internet	Quad-core or higher, 16 GB RAM, modern graphics, 10 GB or more storage, broadband
Administrator computer	Desktop/laptop, 8 GB RAM, modern browser, keyboard, mouse, internet	16 GB RAM, 1920 × 1080 display or higher, broadband
Visitor computer	Modern browser, JavaScript, WebGL, internet	Current browser, hardware acceleration, 8 GB RAM, broadband
Visitor mobile device	Current smartphone/tablet, 4 GB RAM, touchscreen	6 GB RAM or higher, current OS, updated browser, motion sensors
AR/WebXR equipment	Camera permission or supported WebXR device	Camera, motion sensors, compatible headset, and controllers
Capture equipment	Conventional camera and microphone	360-degree camera, tripod, high-resolution camera, external microphone

5. Software Requirements

Table 5.1 Software Requirements

Software	Purpose
Python 3 and Flask 2.3+	Server application, routing, sessions, authentication, and rendering
PyYAML, python-dotenv, Supabase client, Pillow	Configuration, data access, storage, and image validation
Node.js and npm	Preparation of WebXR browser dependencies
HTML5, CSS3, JavaScript, and Jinja	Public and administrative interfaces
Three.js, SparkJS, three-mesh-bvh, WebGL, WebXR	Immersive rendering, splats, movement, and collision
Pannellum and Leaflet	360-degree panoramas and interactive maps
AR.js and Google <model-viewer>	Marker-based AR and three-dimensional model viewing
Supabase PostgreSQL and Storage	Structured records and uploaded multimedia
Git, GitHub, and Vercel	Version control and production deployment

6. Project Folder Structure

```
NHCPSystem/
|-- app.py
|-- requirements.txt
|-- .env
|-- templates/            Public, account, and admin templates
|-- static/               CSS, JavaScript, images, data, and models
|-- public/               Panoramas, AR assets, videos, and WebXR vendor files
|-- database_sql/         Complete schema and migration scripts
|-- webxr_runtime/        Runtime code, scene archives, and Node dependencies
|-- *_content/            Feature routes and services
|-- *_repository.py       Database-access modules
`-- tests/                Automated developer tests
```

> **Important:** Keep the database_sql directory under authorized version control. It is required for rebuilding the database, auditing changes, and applying future migrations.

7. Database and Storage Structure

The fresh-install database is defined in database_sql/AADatabaseSQL.sql. It creates the empty structure, including 28 application tables, constraints, indexes, functions, triggers, foreign keys, and Row Level Security settings. It intentionally inserts no administrator, user, content, booking, or configuration data.

Table 7.1 Database Structure

Database group	Implemented tables	Supported functions	Database group
Administrator security	admin_users, admin_password_reset_requests, admin_password_reset_tokens, admin_login_otp_challenges	Administrator login, login OTP, account status, password recovery, and secure password updating	Administrator security
Administrator notifications	admin_notifications	Persistent administrator notifications for bookings, submissions, and feedback	Administrator notifications
Public-user accounts	public_users, pending_public_regi	Registration, email verification, login,	Public-user accounts

	strations	profile information, preferences, and account status	
User activity and consent	user_bookmarks, public_terms_acceptances, analytics_events	Saved gallery items, acceptance records, and privacy-conscious usage analytics	User activity and consent
Visitor bookings	tour_bookings	Guided-tour requests, administrative review, status tracking, and visitor updates	Visitor bookings
Community content	community_highlights, community_participation_submissions	Published community highlights and visitor or teacher submissions	Community content
Visitor feedback	visitor_feedback, visitor_feedback_moderation_log	Feedback collection, sentiment information, review, publication, and moderation history	Visitor feedback
Online gallery	online_gallery_categories, online_gallery_items	Categorized gallery materials, bilingual descriptions, multimedia, and publication status	Online gallery
Educational resources	edu_resources	Learning materials, documents, media, audience information, and resource publication	Educational resources
Heritage Keeper	heritage_keeper_events, heritage_keeper_awards	Progress events, completed activities, collected fragments, and awarded achievements	Heritage Keeper
WebXR progress	webxr_player_items, webxr_player_museum_views	Collected WebXR objects and viewed museum content	WebXR progress

Virtual tours	virtual_tours, virtual_tour_revisions, virtual_tour_scenes, virtual_tour_hotspots	Virtual-tour environments, drafts, published revisions, scenes, navigation, informational hotspots, coordinates, media, and narration	Virtual tours
Interactive site map	site_maps, site_map_revisions	Aerial and 360-degree map settings, hotspots, related coordinates, bilingual content, audio, images, and destination links	Interactive site map

Virtual Tour and Site Map records use draft and published revisions. An administrator edits a draft and publishes it only after validation. This prevents incomplete configurations from appearing publicly.

7.1 Supabase Database Setup

1. Create or select an empty Supabase project.
2. Open SQL Editor and create a query.
3. Copy the complete contents of database_sql/AADatabaseSQL.sql.
4. Run it and confirm that no error occurs.
5. Verify the tables in Table Editor.

For an existing production project, back up first and apply only the migration required by the compatible application release. Test each migration in staging.

7.2 Storage Buckets

Bucket	Configuration	Purpose
nhcp-media	Public; at least 25 MB per file	Gallery images/GLBs, educational files,

		submissions, avatars, and public media
nhcp-virtual-tours	Private; at least 30 MB per file	Tour panoramas, hotspot media, narration, aerial maps, and 360-degree site maps

If another name is used, set SUPABASE_MEDIA_BUCKET or SUPABASE_VIRTUAL_TOUR_BUCKET. Storage objects require a separate backup from the database.

8. Local Installation

8.1 Obtain the Repository

```
git clone [AUTHORIZED_REPOSITORY_URL]
cd NHCPSystem
```

8.2 Python Environment

Windows:

```
py -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Linux or macOS:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

8.3 WebXR Dependencies

```
npm install --prefix webxr_runtime
```

The command prepares Three.js and three-mesh-bvh under webxr_runtime/static/vendor and SparkJS under public/webxr/vendor. Confirm that webxr_runtime/church.zip, underground.zip, and museum.zip exist.

9. Environment Variables

Create .env in the project root for local development. Never commit this file or any credential.

```
FLASK_ENV=development
FLASK_SECRET_KEY=
SESSION_COOKIE_SECURE=false
ADMIN_SESSION_MINUTES=120

SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
SUPABASE_PUBLISHABLE_KEY=
SUPABASE_MEDIA_BUCKET=
SUPABASE_VIRTUAL_TOUR_BUCKET=

SMTP_HOST=
SMTP_PORT=
SMTP_USERNAME=
SMTP_PASSWORD=
SMTP_USE_TLS=true
SMTP_USE_SSL=false
MAIL_FROM_ADDRESS=
MAIL_FROM_NAME=
ADMIN_PASSWORD_RESET_BASE_URL=

GOOGLE_OAUTH_CLIENT_ID=
GOOGLE_OAUTH_CLIENT_SECRET=
GOOGLE_OAUTH_REDIRECT_URI=
ELEVENLABS_API_KEY=
ELEVENLABS_VOICE_ID=
ELEVENLABS_MODEL_ID=eleven_multilingual_v2
FEEDBACK_SENTIMENT_PROVIDER=
HF_TOKEN=
DEFAULT_QUALITY=auto
DEBUG_COLLISION=false
```

Generate a Flask secret with python -c "import secrets; print(secrets.token_urlsafe(64))". In production use FLASK_ENV=production, HTTPS, and SESSION_COOKIE_SECURE=true. Do not enable both SMTP TLS and SSL. Administrator login requires working email because it sends a six-digit OTP.

9.1 Optional Configuration

Google OAuth requires its client ID, client secret, and exact redirect URI. ElevenLabs

requires an API key and voice ID. Hosted sentiment analysis is opt-in through the Hugging Face provider and token; otherwise the application uses its local method. WebXR quality, movement, collision debugging, and runtime location can be adjusted using the settings documented in webxr_runtime/.env.example.

10. First Administrator Account

The fresh database contains no administrator account. After activating the virtual environment, generate a secure Werkzeug password hash:

```
python
from getpass import getpass
from werkzeug.security import generate_password_hash
password = getpass("Enter the initial administrator password: ")
print(generate_password_hash(password))
exit()
```

Copy the generated hash. In the Supabase SQL Editor, run:

```
INSERT INTO public.admin_users (email, password_hash, is_active)
VALUES (
  'administrator@example.com',
  'PASTE_THE_GENERATED_PASSWORD_HASH_HERE',
  true
);
```

The email must be lowercase and able to receive the administrator OTP. Never store the plain-text password in the database.

11. Local Operation

With the virtual environment active, run python app.py, or use python3 app.py when required. Open http://127.0.0.1:5000. Administrator login is available at http://127.0.0.1:5000/_admin/login. The built-in Flask server is intended only for local development.

During startup, confirm that Flask starts without an exception, Supabase is detected, WebXR initializes, the default scene archive is found, scene metadata is parsed, and vendor resources are available.

11.1 Administrator Login Verification

1. Open /_admin/login.
2. Enter the administrator email and password.
3. Retrieve the six-digit OTP from the registered email.
4. Enter it before expiration.
5. Confirm that the dashboard opens.
6. Open Account Settings and verify the account.

12. Production Deployment

12.1 Vercel Prerequisites

- Commit the approved source and required WebXR resources.
- Install the Supabase schema and create both storage buckets.
- Create a valid administrator account.
- Configure production SMTP and prepare all required environment variables.
- Confirm that no secret is committed to the repository.

12.2 Deployment Procedure

1. Sign in to Vercel and import the authorized Git repository.
2. Select the repository root as the project root.
3. Confirm that Vercel detects app.py as the Flask entry point.
4. Add all required production environment variables.
5. Set FLASK_ENV=production and SESSION_COOKIE_SECURE=true.
6. Set ADMIN_PASSWORD_RESET_BASE_URL to the exact HTTPS production origin without a path.
7. Deploy and review the build and runtime logs.
8. Test the public site, accounts, email, administration, uploads, virtual tour, site map, AR, and WebXR.

12.3 Google OAuth

To enable Google authentication, create a Web Application OAuth client in Google Cloud, configure the consent screen, register the exact callback URI, set the Google OAuth environment variables, redeploy, and test registration and login. The configured redirect must

match exactly in protocol, hostname, port, path, and trailing-slash behavior.

12.4 Post-Deployment Checklist

Verify the landing page, Digital Exhibits, Online Gallery, Educational Resources, Interactive hub, both Site Map views, Virtual Tour, Heritage Trail, AR permission, 3D models, church, museum, and underground WebXR scenes, registration, email verification, password recovery, booking, participation, feedback, administrator OTP, dashboard, uploads, draft saving, validation, and publication.

13. Administrator Functions

13.1 Authentication and Dashboard

The administrator signs in with an email and password and then enters the six-digit OTP delivered by email. Repeated invalid credentials or codes may temporarily restrict login. The dashboard presents summarized users, content, bookings, submissions, feedback, notifications, recent activity, device information, and selected analytics.

13.2 User Management

User Management supports searching and reviewing registered-user accounts, profile information, status, and applicable activity. Personal information must be handled according to approved privacy and retention policies.

13.3 Community Highlights

Administrators can create and edit bilingual highlights, upload images, assign categories, feature eligible content, control publication status, archive records, and perform authorized deletion. Historical accuracy, permission, privacy, and copyright should be checked before publication.

13.4 Booking Calendar

The Booking Calendar displays guided-tour requests and schedule information. Administrators can review and, where supported, confirm, reject, cancel, or reschedule a request and send the corresponding decision to the visitor.

13.5 Online Gallery

Administrators can create gallery items, assign categories, enter bilingual descriptions, upload JPG or PNG images and supported GLB models, save drafts, publish approved records, archive content, and delete authorized records. General gallery media is limited to 25 MB per file.

13.6 Virtual Tour Editor

1. Create or open a virtual-tour environment.
2. Add the scene name, bilingual information, and capture date.
3. Upload and validate a 2:1 equirectangular panorama.
4. Set exactly one starting scene.
5. Set its initial pitch, yaw, and field of view.
6. Add informational and navigational hotspots.
7. Select destination scenes for navigation hotspots.
8. Add images, GLBs, subtitles, and narration where applicable.
9. Configure the upper-right minimap and scene markers.
10. Save the draft and resolve all validation issues.
11. Publish only after the saved draft is complete.

A virtual-tour panorama must be JPG, JPEG, PNG, or WebP, use a 2:1 ratio, measure from 2048 × 1024 through 4096 × 2048 pixels, and remain within 30 MB. Other virtual-tour media is limited to 25 MB.

13.7 Site Map Editor

The editor supports an aerial map, a 360-degree aerial panorama, English and Filipino starting audio, hotspots, aerial coordinates, panorama pitch and yaw, linked aerial/360 representations, bilingual information, categories, marker styles, photographs, alternative text, hotspot narration, destination buttons, virtual-tour scene links, visibility settings, drafts, and publication.

A map image or panorama must remain within 30 MB. A 360-degree map must use a 2:1 equirectangular ratio and measure at least 1024 × 512 pixels. The packaged original is the fallback, while a database revision becomes public only after publication.

13.8 Educational Resources

Administrators can create learning resources, upload a cover, provide descriptions and audience details, attach PDF, DOCX, PPT, or PPTX files, save a draft, publish, archive, and perform authorized deletion. Files must be reviewed before publication.

13.9 Participation and Feedback

Participation management supports review of teacher and community submissions, documents, photographs, verification state, review feedback, approval, rejection, and archival. Feedback management supports moderation, publication, withholding, exclusion, and history. Automated sentiment assists review but does not replace administrator judgment.

13.10 Notifications and Account Settings

The notification panel presents supported bookings, submissions, feedback, and other events. Account Settings permits password maintenance. Changing the password also changes the account version and invalidates older administrator sessions.

14. Backup and Restoration

A complete backup includes the PostgreSQL database, both Supabase Storage buckets, source code, packaged multimedia, WebXR archives, and a protected inventory of production configuration.

Table 14.1 Backup and Restoration

Frequency	Action
Before every deployment	Back up the database and record the current verified deployment
Before a migration	Create and verify a restorable database backup
Daily or by institutional policy	Protect new transactional records and uploaded content

Weekly	Verify database and storage backup completion
Monthly	Test restoration in a separate environment
After major content changes	Back up affected records and storage objects

Back up nhcp-media and nhcp-virtual-tours separately and preserve original object paths. Database exports and secrets must be stored in approved encrypted locations.

14.1 Restoration Procedure

1. Create a staging Supabase project.
2. Restore the database backup.
3. Restore storage objects using their original paths.
4. Configure a staging deployment.
5. Verify accounts, content, bookings, media, virtual tours, site maps, and publication references.
6. Test administrator authentication.
7. Approve the restoration before production use.

15. Routine Maintenance

Table 15.1 Routine Maintenance

Frequency	Task
Daily or frequent	Review Vercel logs, Supabase availability, notifications, bookings, submissions, feedback, email delivery, and the public landing page
Weekly	Test representative gallery/media files, tour and map hotspots, AR, every WebXR scene,

	external links, storage usage, and draft privacy
Monthly	Review administrator access, backups, dependencies, password recovery, OTP, registration, analytics retention, accessibility controls, and representative devices

15.1 Dependency Updates

1. Create a separate Git branch and record existing versions.
2. Update one dependency group at a time.
3. Rebuild WebXR vendor files if applicable.
4. Run automated and manual tests.
5. Deploy to preview or staging first.
6. Release only after verification.

Changes to Three.js, SparkJS, Pannellum, AR.js, or <model-viewer> require renewed testing of immersive features.

15.2 Terms and Conditions

When approved terms materially change, update templates/TermsConditions.html, PUBLIC_TERMS_VERSION, and PUBLIC_TERMS_EFFECTIVE_DATE in legal_content.py. Deploy and verify that users accept the new version. Final wording must be approved by the museum or its legal or data-privacy officer.

16. Security Guidelines

- Never commit environment files or credentials.
- Never expose the Supabase service-role key to the browser.
- Use HTTPS and secure session cookies in production.
- Require administrator OTP and use individual accounts where possible.
- Disable accounts that no longer require access.
- Do not store plain-text passwords.

- Do not bypass authorization, CSRF, rate limiting, or upload validation.
- Review uploaded material before publication.
- Test migrations and destructive operations in staging.
- Do not delete storage objects still referenced by database revisions.
- Rotate exposed Flask, Supabase, SMTP, Google, ElevenLabs, or Hugging Face secrets.

17. Testing and Verification

17.1 Automated Tests

```
python -m unittest discover -s tests -v
```

Use `python3` where required. These developer tests are separate from the 54-case Black Box Testing instrument reported in the research paper.

17.2 Syntax Verification

```
python -m compileall app.py *_content tests
```

17.3 Manual Smoke Test

After major updates, verify public navigation, gallery details, educational files, both Site Map views, representative hotspots, Virtual Tour transitions and narration, all WebXR scenes, AR, public accounts, bookings, feedback, administrator OTP, dashboard records, uploads, draft saving, publication validation, and the resulting public content.

18. Troubleshooting

Table 18.1 Troubleshooting

Problem	Possible Cause	Corrective Action
Application does not start	Dependencies are missing	Activate the virtual environment and install `requirements.txt`

Production refuses to start	Flask secret is missing	Add a strong FLASK_SECRET_KEY
Database content is empty	Supabase configuration or schema is missing	Check environment variables, SQL execution, and logs
Administrator login is unavailable	Supabase is not configured	Verify the URL and service-role key
OTP or verification email does not arrive	SMTP is unavailable or rejected	Check host, port, credentials, security mode, sender, spam, and provider logs
Administrator login is locked	Repeated invalid attempts	Wait for the displayed lockout period
Password reset link is invalid	Wrong trusted origin	Correct ADMIN_PASSWORD_RESET_BASE_URL
Google login fails	Redirect URI mismatch	Match the application and Google callback exactly
Direct upload is unavailable	Publishable key is missing	Set SUPABASE_PUBLISHABLE_KEY
Upload is rejected	Invalid type, size, path, or bucket setting	Use an accepted file and verify bucket limits
Gallery media is missing	Public bucket or object path is incorrect	Verify nhcp-media and the object

Tour/map media is missing	Private bucket or signed URL failed	Verify nhcp-virtual-tours and server access
Panorama is rejected	Wrong ratio, dimensions, format, or size	Use a valid 2:1 equirectangular image within the module limit
Publish is disabled	Changes are unsaved	Save the draft first
Publication fails	Required content is incomplete	Resolve every validation issue and save again
Hotspot is not visible	Visibility or coordinates are incorrect	Verify its views, repin it, save, and publish
WebXR does not load	ZIP, extraction, vendor, browser, or HTTPS problem	Check archives, writable runtime, vendor files, logs, and device support
AR camera does not open	HTTPS or permission is missing	Use HTTPS and allow camera access
3D content is slow	Resource is large for the device	Use lower quality, optimize the model, or use a capable device
Sessions end after deployment	Flask secret changed or cookie settings are inconsistent	Keep the production secret stable and verify HTTPS
Database errors follow an update	Code and schema versions differ	Apply the required migration or restore a compatible release

19. Failure Recovery

1. Stop additional database changes.
2. Record the error and affected release.
3. Review Vercel and Supabase logs.
4. Roll back to the previous verified deployment.
5. Determine whether the failed release modified the database.
6. Restore database or storage only from a verified backup when necessary.
7. Test restoration separately and confirm public and administrator access.
8. Document the incident and corrective action.

Rolling back application code does not automatically reverse a database migration.

20. Technical Limitations

- WebXR depends on browser, device, operating system, permission, and HTTPS support.
- AR requires compatible camera access.
- Large panoramas, videos, and models may load slowly on limited devices or networks.
- WebXR archives are extracted to temporary server storage and may be processed again by a new serverless instance.
- Email, OAuth, narration, document preview, and hosted sentiment depend on external services.
- Database backups do not automatically include Supabase Storage objects.
- Automated sentiment requires administrator review.
- Database-backed Site Map and Virtual Tour edits must be published before becoming public.

21. Technical Handover Checklist

- Repository, Vercel, and Supabase access
- Verified database and storage backups
- Environment-variable inventory and credential-rotation procedure

- SMTP, Google OAuth, ElevenLabs, and Hugging Face access where applicable
- Administrator account and domain/DNS access
- Current deployment URL and version
- Dependency details, pending migrations, known limitations, incidents, and unpublished drafts

Secret values must be transferred through an approved encrypted channel and must not be printed in the publicly submitted research paper.